

Aufgabe 1: AVL-Bäume**(10 Punkte)**

Fügen Sie die folgenden Zahlen nacheinander in einen *AVL-Baum* ein:

110 22 12 33 35 8

Zeichnen Sie den Baum vor und nach jeder durchgeführten Rotation. Geben Sie auch jeweils an, was für Rotationen Sie durchführen.

Aufgabe 2: Heaps**(10 Punkte)**

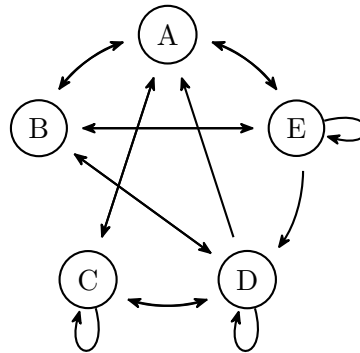
Fügen Sie die folgenden Zahlen nacheinander in einen *Min-Heap* ein:

70 12 30 8 10 1 45

Zeichnen Sie den Baum vor und nach jedem vollständigen Einfügen.

Aufgabe 3: Graphen**(10 Punkte)**

Betrachten Sie den folgenden gerichteten Graphen:



- Erstellen Sie die Adjazenzliste und die Adjazenzmatrix zu diesem Graphen.
- Welche der beiden Darstellungen würden Sie wählen, wenn Sie ...
 - ... häufig überprüfen müssen, ob eine Kante zwischen zwei Knoten existiert?
 - ... häufig über alle Nachbarn eines Knotens iterieren müssten?

Aufgabe 4: Datenstrukturen**(10 Punkte)**

Erklären Sie Idee und Funktionsweise des folgenden Datentyps. Welche Vor- und Nachteile sehen Sie gegenüber einem binären Suchbaum?

```
1  type Table struct {
2      keys    []string
3      values  []string
4  }
5
6  func targetIndex(key string) int {
7      firstchar := key[0]
8      return int((firstchar-'a') * 10)
9  }
10
11 func (h *Table) FirstEmptyIndexForKey(key string) int {
12     for i := 0; i < 260; i++ {
13         index := (targetIndex(key) + i) % 260
14         if h.keys[index] == "" {
15             return index
16         }
17     }
18     return -1
19 }
20
21 func (h *Table) FirstMatchingIndexForKey(key string) int {
22     for i := 0; i < 260; i++ {
23         index := (targetIndex(key) + i) % 260
24         if h.keys[index] == key {
25             return index
26         }
27     }
28     return -1
29 }
30
31 func (h *Table) Put(key string, value string) {
32     index := h.FirstEmptyIndexForKey(key)
33     if index == -1 {
34         panic("Table is full")
35     }
36     h.keys[index] = key
37     h.values[index] = value
38 }
```

Aufgabe 5: Komplexität**(10 Punkte)**

Bestimmen Sie die Komplexität des folgenden Algorithmus zur Bestimmung des Medians einer Liste der Länge n : Geben Sie auch einen Verbesserungsvorschlag an, wie die Komplexität reduziert werden kann.

```
1 func Median(list []int) int {
2     diffs := make([]int, len(list))
3
4     for i, el := range list {
5         d := largerCount(list, el) - smallerCount(list, el)
6         diffs[i] = abs(d)
7     }
8
9     return list[smallestIndex(diffs)]
10 }
```

Gehen Sie dabei davon aus, dass die Hilfsfunktionen tun, was ihre Namen vermuten lassen:

- `largerCount` und `smallerCount` zählen die Anzahl der Elemente, die größer bzw. kleiner als ein gegebenes Element sind.
- `smallestIndex` gibt den Index des kleinsten Elements zurück.
- `abs` berechnet den Betrag einer Zahl.

Gehen Sie weiter davon aus, dass diese Hilfsfunktionen jeweils optimal sind, d.h. dass sie die bestmögliche Laufzeit für ihre Aufgabe haben.